

Universal, Hard-Routed and Soft Mixture-of-Experts Image Restoration and Style-Conditioned Face-to-Sketch Generation: From Research to a Containerised Application

Maryam Fatima

23i-0788, Computer Science, FAST-NUCES

Email: maryam.faatim4@gmail.com

Code: <https://github.com/Maryam-Faatima/genai-restoration-sketch-studio>

Demo video: https://youtu.be/hl0AEWOZavM?si=2v_Tvvj9xiunxTBJ

Abstract—We build and evaluate four related generative systems and deploy them in one browser-based application. On the Oxford-IIIT Pet dataset we study image restoration under three runtime corruptions (salt-and-pepper noise, Gaussian blur, rectangular occlusion): (i) a universal convolutional autoencoder with a genuine bottleneck, (ii) a hard-routed system in which a corruption classifier selects one of three specialist autoencoders, and (iii) a jointly fine-tuned soft mixture-of-experts (MoE) with a differentiable gate and an identity branch. On the FS2K dataset we train (iv) a style-conditioned U-Net/PatchGAN face-to-sketch generator. On a fixed test manifest of 36,690 corrupted images the classifier reaches 99.8% accuracy, hard routing reaches 26.56 dB PSNR / 0.804 SSIM and soft MoE reaches 26.91 dB / 0.828 against 22.34 dB / 0.671 for the unrestored inputs (the overall averages include clean images). The face-to-sketch generator reaches 0.486 SSIM on the official test set and produces visibly blurry sketches, which we analyse. All hyperparameters were tuned with Optuna, experiments were tracked with Weights & Biases, all inference models were exported to ONNX and verified against PyTorch (maximum absolute difference 1.3×10^{-6}), and everything is served through a FastAPI backend and a React frontend started with a single Docker Compose command.

Index Terms—denoising autoencoder, mixture of experts, image restoration, conditional GAN, Optuna, ONNX, FastAPI

I. INTRODUCTION

Real-world images are rarely degraded in one known way. A restoration system must therefore either handle every corruption with one model, decide which specialist to apply, or blend specialists. This work compares these three designs under identical data, splits and metrics, and adds a paired image-to-image generation task. We also treat the project as a software product: every model is exported to ONNX and reachable through one web application.

Our contributions are: (1) a reproducible runtime-corruption pipeline with deterministic validation and test manifests; (2) a documented design history of the universal autoencoder (v1–v3) with controlled ablations; (3) a hard-routed system evaluated with both oracle and predicted routing; (4) a soft MoE initialised from the hard-routed components, with an analysis of its gate; (5) a style-conditioned cGAN with a style

embedding used in both generator and discriminator; and (6) a containerised application whose models match their PyTorch originals.

II. RELATED WORK

Restoration with autoencoders. Denoising autoencoders learn to reconstruct clean inputs from corrupted ones [1]; convolutional denoisers such as DnCNN [2] and inpainting networks [3] extend this to images. Skip connections as in U-Net [4] preserve detail but, when unrestricted, let a network copy its input, which defeats a compression bottleneck. Combining an L_1 term with a structural-similarity term is a well-studied loss for restoration [5], [6].

Mixtures of experts. Mixtures of experts combine specialists through a gating network [7]. Sparse gating at scale [8] and later work [9] motivate auxiliary losses that keep expert usage balanced; we use a differentiable balance term for the same reason.

Conditional generation. GANs [10] and their conditional form [11] underlie paired translation with a U-Net generator and PatchGAN discriminator [12]. Feature-wise linear modulation (FiLM) [13] injects a condition into intermediate features, which we use for the style embedding. FS2K [14] provides photo–sketch pairs in three styles.

Tooling. Hyperparameters were optimised with Optuna [18], experiments tracked with Weights & Biases [19], and models served with ONNX Runtime [20].

III. DATASETS AND EXPERIMENTAL SETUP

A. Oxford-IIIT Pet (Tasks 1–3)

We use the Oxford-IIIT Pet dataset [15]. Images are converted to RGB and resized to 128×128 . The official `trainval` collection (3,680 images) is split 80/20 with random seed 42 into 2,944 training and 736 validation images (no overlap). The official test set (3,669 images) was held out. Class labels are unused: the clean image is the target. The same split is used for Tasks 1–3.

TABLE I
CORRUPTION DEFINITIONS. TEST SEVERITIES ARE FIXED.

Condition	Training (random)	Test (low/med/high)
Clean	none	none
Salt-and-pepper	$p \sim U(0.02, 0.15)$, black/white equally likely	p = 0.03, 0.08, 0.15
Gaussian blur	kernel $\in \{3, 5, 7\}$, $\sigma \sim U(0.5, 2.5)$	(3, 0.7), (5, 1.5), (7, 2.5)
Occlusion	1–3 black rectangles, joint area 10–35%	1, 2, 3 rectangles covering $\approx 10, 20, 35\%$

B. Runtime corruptions

During training the data loader draws, with equal probability, one of four input conditions for every loaded image and samples fresh parameters from operating-system entropy, so no corrupted copies are stored. Table I gives the definitions. The condition label is returned with each sample and trains the classifier of Task 2. Training batches for the classifier and the soft MoE are exactly balanced (verified: [8, 8, 8, 8] per batch of 32).

C. Deterministic manifests

Validation and test corruptions are generated once and stored in JSON manifests (type, severity, blur settings, rectangle coordinates and seed per entry). The validation manifest has 736 entries (exactly 184 per condition). The test manifest has $3,669 \times 10 = 36,690$ entries: each clean test image appears once clean and under each corruption at its three fixed severities. Occlusion rectangles are sampled without overlap so the stated area is respected.

D. FS2K (Task 4)

FS2K [14] contains 2,104 photo–sketch pairs; we use the official split of 1,058 training and 1,046 test pairs. Fifteen percent of the training portion (159 pairs) forms a validation set, stratified by style with seed 42, leaving 899 training pairs (validation: 54/52/53 per style; official test: 619/381/46). Photographs and sketches are resized to 128×128 and pairing is preserved; sketches are grayscale, so the generator outputs one channel. The style label is the `style` field of the annotation file. A cross-tabulation showed that the three photo folders are source subsets, not styles (folder `photo1` mixes styles 0 and 1), so the folder name was not used as the condition. A correlation check supports correct pairing (matched pairs 0.445 versus 0.243 for shifted and 0.207 for shuffled pairs). Spatial augmentation is limited to a horizontal flip applied identically to both images.

E. Metrics, optimisation and tracking

We report $\text{PSNR} = 10 \log_{10}(1/\text{MSE})$ with MSE clamped at 10^{-5} , so PSNR is capped at 50 dB (this cap makes clean images count as 50 dB when no restoration is applied), and SSIM (Gaussian window 11, data range 1). Every table also lists the metrics of the unrestored input as a baseline. “Overall” rows average all conditions, including clean inputs, and should

be read with this cap in mind; the per-condition rows are the meaningful comparison.

All models use AdamW (Task 4: Adam with $\beta_1 = 0.5$), cosine learning-rate schedules and seed 42. Hyperparameters were tuned with Optuna [18] (TPE sampler, seed 42, median pruner, SQLite storage); the objective for restoration tasks is $J = 0.5 \text{ SSIM} + 0.5 \text{ PSNR}/40$ on the validation manifest. Metrics, losses, checkpoints and sample images were logged to Weights & Biases [19]. Training ran on a Tesla T4 GPU in Google Colab.

Test-set protocol. All hyperparameter and checkpoint selection used the validation split only. The official test set was nevertheless evaluated for earlier model versions (v1, v2) for comparison, and the design of v3 was made after seeing those results. No parameter was tuned on test data.

IV. TASK 1: UNIVERSAL DENOISING AUTOENCODER

A. Architecture

A single autoencoder restores clean, noisy, blurred and occluded inputs and is never told the corruption type. The encoder has four stride-2 blocks (each: 4×4 convolution, batch normalisation, LeakyReLU, then a 3×3 convolution with the same activation) taking $128 \times 128 \times 3$ to $8 \times 8 \times 256$ with channels 32, 64, 128, 256. A 1×1 convolution compresses this to a latent of $64 \times 8 \times 8 = 4096$ values, a $12 \times$ reduction of the 49,152 input values, followed by dropout. The decoder mirrors the encoder with four transposed-convolution blocks and ends in a sigmoid. The final model has *no* skip connections (Fig. 1).

B. Loss and training

With clean image x and reconstruction $\hat{x}$ the loss is

$$\mathcal{L} = \alpha \mathcal{L}_1(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x})). \quad (1)$$

The L_1 term drives accurate pixel values and the SSIM term preserves structure. The final model was trained for 100 epochs with batch size 32, learning rate 4.55×10^{-4} , dropout 0.1 and $\alpha = 0.6$ (selected by Optuna).

C. Hyperparameter search and design history

The design evolved over three Optuna studies, each motivated by the previous result (Table II). **v1** used a flattened linear bottleneck (up to 512 values). Its test PSNR was nearly flat across inputs (about 17.4–18.1 dB, even for clean images), indicating that the vector bottleneck, not training length, limited quality: 60 epochs gave only 0.447 against 0.395 for an 8-epoch trial. **v2** replaced the vector by a spatial 8×8 latent that keeps the image layout. **v3** added a narrow skip connection to the search space (width 0, 2, 4 or 8) to test whether limited skips help.

The v3 search space was: learning rate $[5 \times 10^{-5}, 3 \times 10^{-3}]$ (log), batch size $\{16, 32, 64\}$, latent channels $\{16, 32, 48, 64\}$, base channels $\{32, 48, 64\}$, dropout $[0, 0.3]$, $\alpha \in [0.5, 0.95]$ and skip width $\{0, 2, 4, 8\}$. Of 24 trials, 9 completed and 15 were pruned; the best trial (objective 0.6028 after 12 epochs) used latent 64, base 32, $\alpha = 0.6$ and skip width 4. The earlier studies searched a flat bottleneck in $\{64, 128, 256, 512\}$

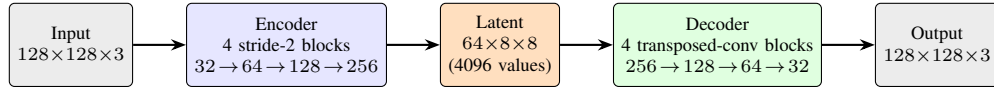


Fig. 1. Task 1 universal autoencoder (final model, no skip connections). The same backbone with a narrow skip option was used in the ablation.

TABLE II
DESIGN HISTORY OF THE UNIVERSAL AUTOENCODER. OPTUNA: TRIALS (COMPLETED/PRUNED) $\times$ EPOCHS PER TRIAL. TEST METRICS ARE OVERALL MEANS OVER 36,690 TEST CONDITIONS.

Version	Bottleneck	Optuna	Final epochs	Val. score	Test PSNR / SSIM
v1	flat vector (512)	20 (8/12) $\times$ 8	60	0.447	17.85 / 0.439
v2	spatial $8 \times 8 \times 32$	20 (8/12) $\times$ 8	40	0.584	20.52 / 0.639
v3, skip 4	spatial $8 \times 8 \times 64$ + narrow skip	24 (9/15) $\times$ 12	100	0.639	not evaluated
v3, no skip (final)	spatial $8 \times 8 \times 64$	(same study)	100	0.646	21.63 / 0.735

Missing figure: optuna/task1_ae_v3_history.png
Missing figure: optuna/task1_ae_v3_importance.png

Fig. 2. Task 1 (v3) Optuna study: optimisation history (left) and parameter importance (right).

Missing figure: curves/task1_val_score.png

Fig. 3. Task 1 final training: validation score versus epoch for the skip and no-skip runs.

TABLE III
15-EPOCH ABLATION OF LATENT WIDTH AND NARROW SKIP (VALIDATION SCORE J ; ONE SEED PER ROW).

Run	A	B	C	D
Latent channels	32	64	32	32
Skip width	0	0	4	8
Validation score J	0.575	0.569	0.606	0.610

(v1) and latent channels $\{4, 8, 16, 32\}$ (v2). Figure 2 shows the optimisation history and parameter importances of the v3 study.

D. Are skip connections needed? A controlled ablation

The assignment permits limited skips if investigated. In 15-epoch runs with otherwise identical settings, a narrow skip helped (Table III): width 4 and 8 reached 0.606 and 0.610 against 0.575 and 0.569 without a skip, and widening the latent alone (64 vs. 32 channels) did not help. However, short runs favour skips, which speed up early learning. We therefore trained the Optuna-best configuration for 100 epochs with and without the skip: the no-skip model scored **0.646** (SSIM 0.745, PSNR 21.88 dB) and the skip model 0.639 (SSIM 0.734, PSNR 21.75 dB). The gap is small and based on one seed each, so we do not claim the skip hurts, only that it gives no benefit at convergence. We chose the **no-skip model** as the final Task 1 model: it is at least as accurate, simpler, and satisfies the bottleneck requirement without needing justification. Skip width 4 adds 1,024 values (a $16 \times 16 \times 4$ map), well below the 4,096-value latent.

E. Results

Table IV reports the final model on the official test set, separately per condition and severity, next to the unrestored

TABLE IV
TASK 1 TEST RESULTS (FINAL NO-SKIP MODEL) VERSUS THE UNRESTORED INPUT.

Condition	n	PSNR _{in}	PSNR _{out}	SSIM _{in}	SSIM _{out}
Overall	36690	22.34	21.63	0.671	0.735
Clean	3669	50.00	22.45	1.000	0.775
Salt-and-pepper	11007	16.38	22.34	0.382	0.760
Low	3669	20.14	22.44	0.603	0.771
Medium	3669	15.87	22.37	0.339	0.762
High	3669	13.14	22.20	0.204	0.745
Gaussian blur	11007	27.82	22.33	0.814	0.753
Low	3669	32.34	22.54	0.944	0.774
Medium	3669	26.77	22.49	0.806	0.765
High	3669	24.35	21.97	0.692	0.721
Occlusion	11007	13.59	19.96	0.709	0.678
Low	3669	16.68	21.11	0.862	0.730
Medium	3669	13.34	20.05	0.727	0.686
High	3669	10.74	18.72	0.538	0.619

input. Qualitative results appear in Fig. 4–5 (12 examples: clean target, corrupted input, reconstruction and absolute-error map) and Fig. 6.

Interpretation. The universal model clearly helps for salt-and-pepper noise (SSIM 0.382 $\rightarrow$ 0.760 and PSNR 16.4 $\rightarrow$ 22.3 dB, with 0.204 $\rightarrow$ 0.745 SSIM at the highest severity). Its output quality is almost independent of the input: about 22.2–22.5 dB for clean inputs and for all three noise severities. This is the signature of a capacity ceiling set by the bottleneck, not of condition-specific restoration. The cost is visible on inputs that need no restoration: clean images drop to 22.45 dB and mild blur drops from 32.34 to 22.54 dB, so for Gaussian blur the output is below the input in both PSNR and SSIM at every severity. Occlusion is the only condition where output quality depends on severity (PSNR 21.11 dB at low severity versus 18.72 dB at high); PSNR improves over the input (13.59 $\rightarrow$ 19.96 dB), but overall SSIM is lower (0.709 $\rightarrow$ 0.678)

because unaffected regions are blurred. Compared with the design history (Table II), moving from the vector to the spatial latent raised test SSIM from 0.439 to 0.639, and the final model reached 0.735. These limits motivate Tasks 2 and 3, where clean inputs bypass restoration.

F. Failure cases

Figure 6 shows the worst test image per condition.

- 1) **Clean input, PSNR 16.2 dB, SSIM 0.41.** A dog in a flower field: dense, high-frequency texture cannot be stored in the latent, so the reconstruction loses detail and colour and comes out blurry and sepia-toned.
- 2) **Salt-and-pepper (high), SSIM 0.38, and Gaussian blur (high), SSIM 0.35.** The same image is the worst case for the clean, noisy and blurred inputs. The failure is therefore driven by image content (texture), not by the corruption type.
- 3) **Occlusion (high), PSNR 11.3 dB, SSIM 0.26.** A cat on a black background. Black masks are indistinguishable from the background, so the model cannot tell which dark regions are missing content; it fills the hole with a grey haze that spills into the empty background, and the error map is brightest there.

G. ONNX export and application

The final model is exported to ONNX (10.2 MB; maximum absolute output difference to PyTorch 1.06×10^{-6} ; 18 ms per image on CPU, Section VIII) and available in the application under *Universal Restoration*.

V. TASK 2: CORRUPTION CLASSIFICATION AND HARD-ROUTED SPECIALISTS

A. System overview

Instead of asking one autoencoder to solve every restoration problem, a CNN classifier predicts the corruption type and a matching specialist autoencoder restores the image (Fig. 7). A clean input uses an identity bypass and is never processed by a restoration expert:

$$\hat{x} = \begin{cases} x_c, & k = \text{clean}, \\ E_k(x_c), & k \in \{\text{salt}, \text{blur}, \text{occ}\}, \end{cases} \quad k = \arg \max_i p_i. \quad (2)$$

B. Corruption classifier

The classifier has four blocks (two 3×3 convolutions with batch normalisation [16] and ReLU, then 2×2 max-pooling) with channels 16, 32, 64, 128, followed by concatenated global average and global max pooling (256 features), dropout 0.5 and a linear layer with four outputs (295,028 parameters). Max pooling helps detect isolated noise pixels and black rectangles. It is trained with multiclass cross-entropy on labels produced automatically by the runtime corruption pipeline. Batches are exactly balanced: the class of sample i is $i \bmod 4$ and shuffling is disabled, so every batch of 32 holds eight images per class (verified by printing the first batch).

TABLE V
CLASSIFIER TEST METRICS (OFFICIAL TEST SET, FIXED MANIFEST).

Class	Precision	Recall	F1	Support
Clean	0.995	0.985	0.990	3669
Salt-and-pepper	1.000	1.000	1.000	11007
Gaussian blur	0.995	1.000	0.998	11007
Occlusion	1.000	0.998	0.999	11007
Macro average	0.998	0.996	0.997	36690

Overall accuracy: 0.9980.

Optuna. The search space was learning rate $[3 \times 10^{-4}, 3 \times 10^{-3}]$ (log), batch size $\{16, 32, 64\}$, channel configuration $\{\text{small}, \text{medium}, \text{large}\}$, dropout $[0, 0.5]$ and weight decay $[10^{-6}, 10^{-2}]$ (log), with validation macro-F1 as the objective. Of 30 trials, 6 completed and 24 were pruned; the best validation macro-F1 was 0.9986 with learning rate 7.1×10^{-4} , batch size 16, the *small* configuration, dropout 0.5 and weight decay 2.5×10^{-4} . The final model was trained for 40 epochs (validation macro-F1 1.000, accuracy 1.000).

Test results. On the 36,690 test conditions the classifier reaches an accuracy of 99.80% and a macro-F1 of 0.997 (Table V, Fig. 9). All errors are 72 images: 51 clean images routed to blur, 3 clean images routed to occlusion and 18 occluded images routed to the clean bypass. Clean has the lowest recall (0.985): smooth, low-texture clean photographs resemble blur, and clean photographs with large black borders or backgrounds resemble masks.

C. Specialist autoencoders

Each specialist uses the Task 1 backbone (about 2.5M parameters, 10.1 MB as ONNX) and is trained only on its own corruption with the same clean target. A shared Optuna search identified a common architecture on a proxy task (one autoencoder trained on the union of the three corruptions), after which the three specialists were trained independently with their own data, seed (42 plus the class index) and parameters. The search space was learning rate $[5 \times 10^{-5}, 2 \times 10^{-3}]$, batch size $\{16, 32, 64\}$, base channels $\{32, 48, 64\}$, latent channels $\{16, 32, 48, 64\}$, skip width $\{0, 2, 4, 8\}$ and $\alpha \in [0.5, 0.95]$. Of 12 trials, 6 completed and 6 were pruned; the best (objective 0.5999) used learning rate 9.0×10^{-4} , batch size 16, base 32, latent 48 channels (8×8 , 3072 values), skip width 8 (a $16 \times 16 \times 8$ map, 2048 values, below the latent size) and $\alpha = 0.6$. The four best completed trials all used skip width 8 and the two completed trials without a skip scored lowest. Because short trials favour skips (Section IV-C), this evidence is weaker than the Task 1 control, and we did not run a no-skip control for the specialists. Each specialist was trained for 60 epochs.

Design history. As in Task 1, the specialists were re-designed twice (Table VI). With the flat bottleneck (v1), oracle-routed output quality was flat at about 17 dB for every corruption. The spatial latent (v2) and then the narrow skip with a lower α (v3) raised overall oracle SSIM from 0.468 to 0.715 to 0.804. The v1 search chose $\alpha = 0.95$ (almost pure

Missing figure:
task1/examples_1.png

Fig. 4. Task 1 examples (1/2): rows show clean target, corrupted input, reconstruction and absolute error (test set, fixed manifest).

Missing figure:
task1/examples_2.png

Fig. 5. Task 1 examples (2/2).

Missing figure:
task1/failure_cases.png

Fig. 6. Task 1 failure cases (worst SSIM per condition): clean, salt-and-pepper (high), blur (high), occlusion (high).

L_1 , which favours blurry output), and the v3 search chose $\alpha = 0.6$; since architecture and loss weight changed together, we cannot attribute the gain to either alone.

Validation results. On their own validation images (184 each), the final specialists give: salt-and-pepper PSNR 16.00→25.99 dB and SSIM 0.348→0.834; blur 29.00→26.19 dB and 0.852→0.828; occlusion 13.22→20.90 dB and 0.694→0.699 (input → output).

D. Oracle versus predicted routing

Oracle routing selects the expert from the true label stored in the test manifest and shows the ability of the specialists; predicted routing uses the classifier and measures the full operational system (Table VII).

Interpretation. Predicted routing is almost identical to oracle routing (overall 26.56 versus 26.60 dB, SSIM 0.804 for both), because the classifier is nearly perfect. The specialists help most for salt-and-pepper noise (SSIM 0.382→0.827; at the highest severity 0.204→0.822, with a flat 25.5–25.6 dB across severities). Hard routing beats the universal autoencoder of Task 1 in every condition (salt 25.57 versus 22.34 dB, blur 25.36 versus 22.33 dB, occlusion 21.07 versus 19.96 dB). Two weaknesses remain. For Gaussian blur the specialist is *below the input* at low severity (25.69 versus 32.34 dB and SSIM 0.828 versus 0.944) and only exceeds it at high severity (24.61 versus 24.35 dB, SSIM 0.757 versus 0.692). For occlusion the PSNR gain is large (+7.5 dB) but SSIM is only level with the input overall (0.716 versus 0.709) and below it at low severity (0.769 versus 0.862): the specialist fills the masked region but blurs the untouched pixels. A hard choice between “process fully” and “do nothing” cannot represent mild corruption, which motivates Task 3.

E. Classifier errors and restoration failures

Table VIII lists the 72 misrouted images. Sending a clean image to a specialist is costly because a restoration expert damages an image that needed no work (mean SSIM drop 0.164 for blur, 0.351 for occlusion; in the worst cases SSIM falls from 1.00 to 0.54–0.58, Fig. 11). The two worst images routed to occlusion are a cat on a black background and an image with black letterbox bars: dark regions look like masks

and the occlusion specialist replaces them with a haze (bright error at the borders). The two routed to blur have soft, out-of-focus backgrounds and are re-textured by the blur specialist. In contrast, the 18 occluded images routed to the clean bypass had a negative drop (−0.088): leaving them unprocessed gave a *higher* SSIM than the occlusion specialist, again showing that the specialist hurts SSIM. Across the whole test set the 72 errors reduce overall PSNR by only 0.04 dB, but they are the largest single-image failures of the system.

The classifier and the three specialists are exported to ONNX (Section VIII) and available in the application as *Hard-Routed Restoration*.

VI. TASK 3: JOINTLY TRAINED SOFT MIXTURE-OF-EXPERTS

A. Architecture

A hard decision is brittle near class boundaries and cannot express mild corruption. The soft MoE keeps the four branches of Task 2 (identity, three specialists) but replaces the arg-max with a continuous weighting by a gating network g (Fig. 12):

$$w = \text{softmax}\left(\frac{g(x_c)}{T}\right), \quad \hat{x} = w_c x_c + w_s E_{\text{salt}}(x_c) + w_b E_{\text{blur}}(x_c) + w_o E_{\text{occlusion}}(x_c) \quad (3)$$

The mixture is differentiable, so gate and experts are trained jointly through the final reconstruction error. The gate is initialised from the Task 2 classifier, whose four outputs are already ordered as (clean, salt, blur, occlusion), and the experts from the three trained specialists; nothing is initialised randomly.

B. Loss and training schedule

The joint loss combines reconstruction, structure, classification and a routing-balance term:

$$\mathcal{L} = \lambda_1 \mathcal{L}_1 + \lambda_2 (1 - \text{SSIM}) + \lambda_3 \mathcal{L}_{CE} + \lambda_4 \sum_{i=1}^4 \left(\bar{w}_i - \frac{1}{4}\right)^2, \quad (4)$$

with $\lambda_1 = \alpha$, $\lambda_2 = 1 - \alpha$, $\lambda_3 = \lambda_{cls}$ and $\lambda_4 = \lambda_{bal}$. The cross-entropy is computed on the tempered logits g/T , so it constrains the same weights the mixture uses, and $\bar{w}_i$ is the mean weight of branch i in an exactly balanced batch (target 1/4 each, as in switch-style load balancing [8], [9]). Training has two phases. In a 3-epoch **warm-up** the experts are frozen and kept in evaluation mode (fixed batch-normalisation statistics) and only the gate is trained at learning rate 5×10^{-4} . In 20 **joint** epochs all parameters are fine-tuned at the smaller learning rate η with cosine decay. A trial is

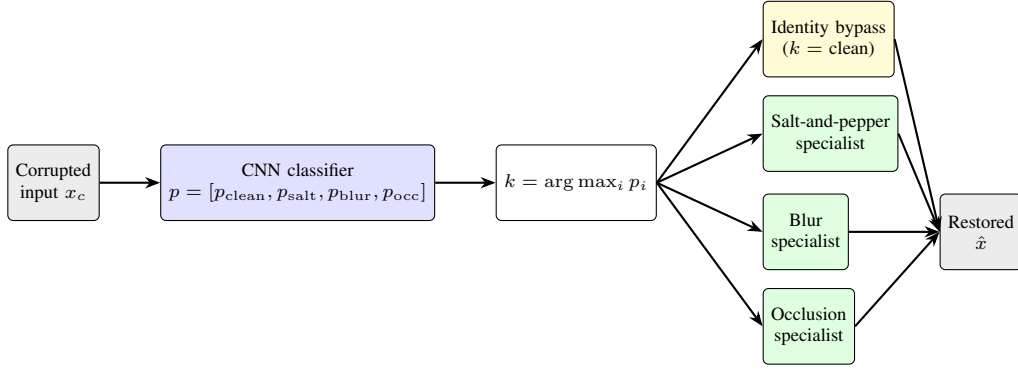


Fig. 7. Hard-routed restoration. The classifier selects exactly one branch; clean inputs are passed through unchanged.

TABLE VI

SPECIALIST DESIGN HISTORY. OPTUNA: TRIALS (COMPLETED/PRUNED) $\times$ EPOCHS PER TRIAL; ONE TRIAL IN V1 WAS INTERRUPTED. TEST RESULTS USE ORACLE ROUTING.

Version	Optuna search	Best obj.	Final ep.	Overall PSNR / SSIM	Salt SSIM	Blur SSIM	Occ. SSIM
v1 (flat vector)	13 (4/8/1 interrupted) $\times$ 6	0.390	40	20.71 / 0.468	0.414	0.412	0.400
v2 (spatial latent)	8 (4/4) $\times$ 6	0.537	30	24.10 / 0.715	0.713	0.716	0.622
v3 (final)	12 (6/6) $\times$ 8	0.600	60	26.60 / 0.804	0.827	0.803	0.716

TABLE VII

TASK 2 TEST RESULTS: UNRESTORED INPUT, ORACLE ROUTING AND PREDICTED ROUTING, AND CLASSIFIER ACCURACY PER GROUP.

Condition	PSNR _{in}	PSNR _{ora}	PSNR _{pred}	SSIM _{in}	SSIM _{ora}	SSIM _{pred}	Cls. acc.
Overall	22.34	26.60	26.56	0.671	0.804	0.804	0.998
Clean	50.00	50.00	49.63	1.000	1.000	0.997	0.985
Salt-and-pepper	16.38	25.57	25.57	0.382	0.827	0.827	1.000
Low	20.14	25.63	25.63	0.603	0.831	0.831	1.000
Medium	15.87	25.59	25.59	0.339	0.828	0.828	1.000
High	13.14	25.47	25.47	0.204	0.822	0.822	1.000
Gaussian blur	27.82	25.36	25.36	0.814	0.803	0.803	1.000
Low	32.34	25.69	25.69	0.944	0.828	0.828	1.000
Medium	26.77	25.78	25.78	0.806	0.823	0.823	1.000
High	24.35	24.61	24.61	0.692	0.757	0.757	1.000
Occlusion	13.59	21.06	21.07	0.709	0.716	0.716	0.998
Low	16.68	22.40	22.42	0.862	0.769	0.769	0.995
Medium	13.34	21.16	21.16	0.727	0.724	0.724	1.000
High	10.74	19.62	19.62	0.538	0.655	0.655	1.000

Missing figure: optuna/task2_cls_history.png
Missing figure: optuna/task2_cls_importance.png

Fig. 8. Classifier Optuna study: optimisation history and parameter importance.

Missing figure: optuna/task2_spec_v3_history.png
Missing figure: optuna/task2_spec_v3_importance.png

Fig. 10. Specialist (v3) Optuna study: history and parameter importance.

Missing figure: task2/confusion_matrix.png

Fig. 9. Normalised four-class confusion matrix on the test set.

pruned if any branch’s mean validation weight falls below 0.02 (routing collapse); none occurred.

C. Optuna search

The search covered $\eta \in [10^{-5}, 2 \times 10^{-4}]$ (log), $T \in [0.5, 3]$, $\lambda_{cls} \in [0.01, 1]$ (log), $\lambda_{bal} \in [10^{-3}, 0.5]$ (log) and $\alpha \in [0.5, 0.95]$, with the validation objective of Section III-E and 1 warm-up plus 4 joint epochs per trial. All 10 trials completed (none pruned) and their objectives lay between 0.7998 and 0.8114, so the model is insensitive to these hyperparameters. The best trial used $\eta = 1.13 \times 10^{-4}$, $T = 1.26$, $\lambda_{cls} = 0.016$,

Missing figure:
task2/misrouting_failures.png

Fig. 11. Restoration failures caused by classifier errors (clean images routed to occlusion and blur). Rows: clean target, input, oracle routing, predicted routing, absolute error.

TABLE VIII
MISROUTED TEST IMAGES AND THE SSIM CHANGE RELATIVE TO ORACLE ROUTING (POSITIVE = WORSE).

True class	Routed to	n	Mean SSIM drop
Clean	Blur	51	0.164
Clean	Occlusion	3	0.351
Occlusion	Clean (bypass)	18	-0.088
All misrouted		72	0.109

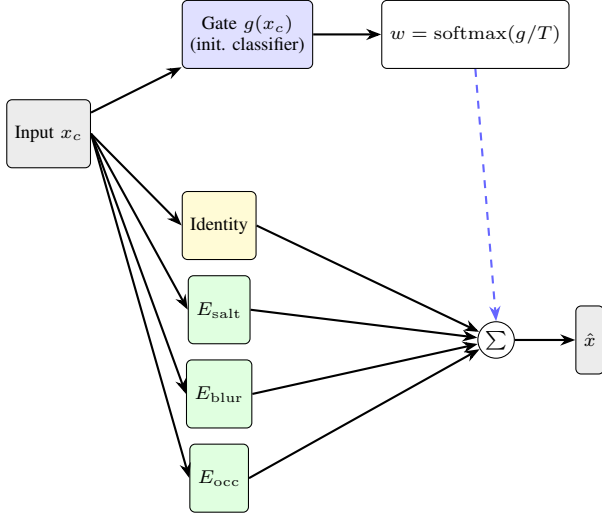


Fig. 12. Soft mixture-of-experts. The dashed arrow carries the four routing weights to the weighted sum.

$\lambda_{bal} = 0.070$ and $\alpha = 0.7$. The final model reached its best validation score (0.8164) at joint epoch 16 of 20.

D. Reconstruction results

The soft MoE improves overall quality over hard routing (26.91 versus 26.56 dB and SSIM 0.828 versus 0.804). The gain is concentrated where hard routing was weakest, at *low severity*: blur/low improves from 25.69 to 29.18 dB and 0.828 to 0.905 SSIM, and occlusion/low SSIM from 0.769 to 0.865 (equal to the input, 0.862). At high severity soft and hard routing are nearly equal (blur 0.758 versus 0.757, occlusion 0.663 versus 0.655, salt 0.820 versus 0.822). The soft model is slightly *worse* in PSNR for salt-and-pepper (25.25 versus 25.57 dB, equal SSIM), and even the best system stays below the input for the mildest blur (29.18 versus 32.34 dB, SSIM 0.905 versus 0.944). Table X summarises all systems.

E. Gate behaviour

Table XI gives the mean routing weights per true condition and severity, and Fig. 14 shows them as a heatmap. The gate learned a *severity-aware* blend. Clean images receive

Missing figure:
optuna/task3_moe_history.png

Missing figure:
optuna/task3_moe_importance.png

Fig. 13. Soft-MoE Optuna study: history and parameter importance.

Missing figure:
task3/routing_heatmap.png

Fig. 14. Mean routing weights by true condition and severity.

97% identity weight and salt-and-pepper noise goes almost entirely to the salt expert (0.994 and 1.000 at medium and high severity). For blur and occlusion the weight moves from a nearly even identity/expert split at low severity (0.482/0.507 and 0.485/0.514) to expert-dominated at high severity (blur 0.905, occlusion 0.939). This is the behaviour that explains the low-severity gains: when the corruption is mild, the identity branch is the better predictor, and the gate has learned to use it.

Dominance and inactive experts. No expert is inactive (Table XII). Mean weights are 0.253 (identity), 0.290 (salt), 0.229 (blur) and 0.229 (occlusion), and each branch wins the arg-max for a share of images that matches its true share (the salt expert wins 29.4% of images against a 30.0% share of salt-and-pepper inputs). The experts are specific: on non-matching conditions the salt, blur and occlusion experts receive on average only 0.000, 0.002 and 0.007. Only the identity branch spreads onto unrelated inputs (mean 0.173 on corrupted images), and, as shown above, this is deliberate use for mild corruption, not collapse. For 50.6% of images one branch has a weight above 0.95 (Fig. 15); for 8.8% the largest weight is below 0.60 and the weights are spread across experts (Fig. 15).

Gate accuracy. The arg-max of the weights agrees with the true class for 88.4% of test conditions, against 99.8% for the Task 2 classifier. During training the validation gate accuracy fell from 0.996 (after one warm-up epoch) to 0.985 (end of warm-up), 0.946 (first joint epoch) and 0.940 (best epoch), while the mean identity weight on validation rose from 0.27 to 0.36 (the validation set is 25% clean). The gate trades classification accuracy for reconstruction quality, as expected from a small λ_{cls} of 0.016: it mixes identity into mild corruptions, so its arg-max no longer equals the true label, yet the restoration is better. The balance term rose from 0.003 to about 0.014 for the same reason (identity receives more than the 1/4 target); the regulariser's weight of 0.070 was enough to prevent any collapse without forcing uniform usage.

F. Failure cases

Figure 16 shows the worst test image per condition.

TABLE IX
TASK 3 TEST RESULTS: UNRESTORED INPUT, HARD ROUTING WITH THE CLASSIFIER (TASK 2) AND THE SOFT MOE.

Condition	PSNR _{in}	PSNR _{hard}	PSNR _{soft}	SSIM _{in}	SSIM _{hard}	SSIM _{soft}
Overall	22.34	26.56	26.91	0.671	0.804	0.828
Clean	50.00	49.63	49.74	1.000	0.997	0.999
Salt-and-pepper	16.38	25.57	25.25	0.382	0.827	0.829
Low	20.14	25.63	25.51	0.603	0.831	0.840
Medium	15.87	25.59	25.16	0.339	0.828	0.827
High	13.14	25.47	25.07	0.204	0.822	0.820
Gaussian blur	27.82	25.36	26.89	0.814	0.803	0.831
Low	32.34	25.69	29.18	0.944	0.828	0.905
Medium	26.77	25.78	26.51	0.806	0.823	0.831
High	24.35	24.61	24.98	0.692	0.757	0.758
Occlusion	13.59	21.07	20.97	0.709	0.716	0.766
Low	16.68	22.42	21.91	0.862	0.769	0.865
Medium	13.34	21.16	21.28	0.727	0.724	0.769
High	10.74	19.62	19.73	0.538	0.655	0.663

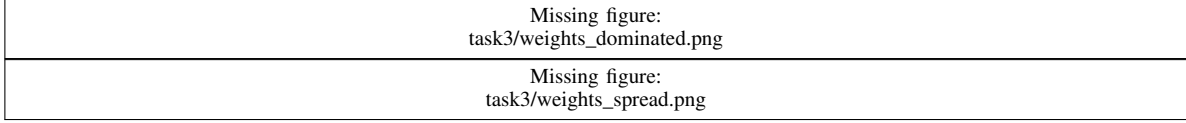


Fig. 15. Left: examples where one expert dominates ($w > 0.95$). Right: examples where weights are spread across branches.

TABLE X
ALL SYSTEMS ON THE TEST SET: PSNR (dB) / SSIM PER CONDITION.

Condition	Input	Universal	Hard	Soft
Overall	22.34 / 0.671	21.63 / 0.735	26.56 / 0.804	26.91 / 0.828
Clean	50.00 / 1.000	22.45 / 0.775	49.63 / 0.997	49.74 / 0.999
Salt-and-pepper	16.38 / 0.382	22.34 / 0.760	25.57 / 0.827	25.25 / 0.829
Gaussian blur	27.82 / 0.814	22.33 / 0.753	25.36 / 0.803	26.89 / 0.831
Occlusion	13.59 / 0.709	19.96 / 0.678	21.07 / 0.716	20.97 / 0.766

TABLE XI
MEAN GATE WEIGHTS BY TRUE CONDITION AND SEVERITY (TEST SET).

Condition	Identity	Salt	Blur	Occ.
Clean	0.971	0.000	0.011	0.018
Salt, low	0.078	0.905	0.000	0.017
Salt, medium	0.004	0.994	0.000	0.001
Salt, high	0.000	1.000	0.000	0.000
Blur, low	0.482	0.000	0.507	0.010
Blur, medium	0.134	0.000	0.863	0.003
Blur, high	0.093	0.000	0.905	0.002
Occlusion, low	0.485	0.000	0.001	0.514
Occlusion, medium	0.217	0.000	0.000	0.782
Occlusion, high	0.061	0.000	0.000	0.939

- 1) **Clean (SSIM 0.450)**: a clean cat on a black background receives weights (identity 0.16, occlusion 0.84), so the occlusion expert damages an image that needed no restoration, the same image type that fooled the Task 2 classifier.
- 2) **Salt-and-pepper, medium (SSIM 0.412)**: the gate gives identity 0.61 and the salt expert only 0.26, so much of the noise passes through; the gate misjudged the severity.
- 3) **Blur, high (SSIM 0.403)**: weights (identity 0.27, blur

TABLE XII
GATE HEALTH ON THE TEST SET.

Branch	Mean weight	Arg-max share	Mean weight on other types
Identity	0.253	0.216	0.173
Salt	0.290	0.294	0.000
Blur	0.229	0.244	0.002
Occlusion	0.229	0.245	0.007

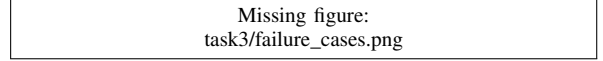


Fig. 16. Task 3 failure cases (worst SSIM per condition) with their gate weights.

0.72) leave part of the blurred input untouched, a mixture that the blur expert alone would have restored better.

- 4) **Occlusion, high (SSIM 0.260)**: the same black-background cat image as in Tasks 1 and 2; the gate is confident (occlusion 0.99), but the expert cannot recover content hidden under a mask that is indistinguishable from the background.

The complete pipeline (gate, three experts, softmax and weighted sum, with $T = 1.26$ fixed) is exported as one ONNX graph that returns the restored image and the four weights, and is available as *Soft Mixture-of-Experts Restoration*.

VII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH GENERATION

A. Architecture

The generator is a U-Net that maps a photograph x and a style $c \in \{1, 2, 3\}$ to a one-channel sketch $\hat{s} = G(x, c)$.

Missing figure: optuna/task4_gan_history.png
Missing figure: optuna/task4_gan_importance.png

Fig. 17. Task 4 Optuna study: history and parameter importance.

A learned embedding of the style (32 dimensions) is used in two ways: it is broadcast over the image and concatenated to the input as extra channels, and it drives feature-wise linear modulation [13] in every decoder block (a per-channel scale and shift predicted from the embedding, initialised to the identity). The encoder has six stride-2 blocks (base 48 channels, widths $b, 2b, 4b, 8b, 8b, 8b$ down to a 2×2 map) and the decoder five transposed-convolution blocks with skip connections, dropout in the first three, and a final transposed convolution with $\tanh$ (about 16.5 M parameters). The Patch-GAN discriminator receives the photograph, the sketch and its own style embedding (broadcast as channels) and outputs a 14×14 map of patch logits, so the style condition is part of both networks and not only an interface label.

B. Objective and training

With binary cross-entropy on logits, the discriminator minimises $\frac{1}{2}(\mathcal{L}_{real} + \mathcal{L}_{fake})$ where real pairs (x, s, c) are labelled 1 and generated pairs $(x, G(x, c), c)$ are labelled 0, and the generator minimises

$$\mathcal{L}_G = \mathcal{L}_{adv} + \lambda_{L1} \mathcal{L}_1(s, G(x, c)). \quad (5)$$

We use Adam [17] with $\beta = (0.5, 0.999)$ and separate learning rates for G and D . The discriminator real and fake losses, the generator adversarial and L_1 losses and the validation metrics (L1, PSNR, SSIM and a style-sensitivity measure) are logged separately each epoch, and a fixed set of validation photographs is rendered with the true style and all three styles at regular intervals (to Weights & Biases) so progress can be compared over time.

C. Hyperparameter search

The space was generator learning rate $[10^{-4}, 10^{-3}]$, discriminator learning rate $[5 \times 10^{-5}, 5 \times 10^{-4}]$, batch size $\{8, 16, 32\}$, base channels $\{32, 48, 64\}$, dropout $[0, 0.5]$, style-embedding dimension $\{8, 16, 32\}$ and $\lambda_{L1} \in \{10, 25, 50, 100, 200\}$, with validation SSIM as the objective and 15 epochs per trial. Of 12 trials, 4 completed and 8 were pruned. The best (objective 0.5006) had $\eta_G = 5.96 \times 10^{-4}$, $\eta_D = 4.35 \times 10^{-4}$, batch size 32, base 48, dropout 0.1, embedding dimension 32 and $\lambda_{L1} = 200$; it was retrained for the full 100 epochs (best validation SSIM 0.508, PSNR 16.14 dB, L1 0.095).

D. Results

On the official test set (1,046 pairs, never used during training or tuning) the generator reaches L1 0.109, PSNR 15.27 dB and SSIM 0.486 (Table XIII). Style 2 (the darkest, heavy-stroke style) is clearly the hardest (12.38 dB); the 46

TABLE XIII
TASK 4 TEST RESULTS (OFFICIAL TEST SET).

Group	n	L1	PSNR	SSIM
Overall	1046	0.109	15.27	0.486
Style 1	619	0.085	16.84	0.534
Style 2	381	0.153	12.38	0.394
Style 3	46	0.071	18.01	0.597

style-3 test images are too few for a reliable estimate, because the official test split has 619, 381 and 46 images of the three styles.

Does the style condition matter? Generating the same photograph under different styles changes the output by a mean absolute difference of 0.102 (on a $[0, 1]$ scale). The output generated with the true style is closest to the ground-truth sketch for 81.1% of the test images; the fair baseline for this statistic is not the 33.3% of a uniform guess but the 59.2% share of style 1, which an always-lightest generator might approach, so 81.1% is a moderate but real signal. Visually (Fig. 18) the style changes mostly the stroke weight and contrast: Style 1 is the lightest, Style 2 the darkest with heavy hair and contour strokes, and Style 3 faint and low-contrast, consistent with the real sketches.

E. Limitations and an ablation on the reconstruction weight

The generated sketches are soft and blurry: they capture the head shape, hair mass, eyes and mouth but not the line character (individual strokes, hatching) of the real sketches, and a faint grid texture is visible in dark areas, consistent with transposed-convolution upsampling. The elderly subject in the third style comparison is rendered much more faintly than the others, and Style 2 turns his features into a messy blur. We tested whether the large reconstruction weight, which encourages averaging, was the cause by retraining the best configuration with $\lambda_{L1} = 50$ (Table XIV). Test metrics were unchanged within run-to-run noise (SSIM 0.486 versus 0.484), and the sketches looked equally soft, so the hypothesis is *not supported*. We kept $\lambda_{L1} = 200$, which was selected by the search. The training logs suggest other causes. The discriminator losses settle near 0.5 (real 0.52, fake 0.51 at epoch 100), so the adversarial signal is modest. Training L1 falls from 0.298 to 0.113 while validation L1 stays near 0.100 from epoch 10 on, and validation SSIM peaks around epoch 20 (0.508) before drifting down to 0.469, so the model stops improving on unseen faces early; we therefore keep the best-validation checkpoint. (Training L1 is measured with dropout active, so it is not directly comparable to validation L1.) The hand-drawn sketches are also only loosely aligned with the photographs, which penalises sharp lines under pixel-wise losses. These are hypotheses supported by the logs; we did not isolate them with further experiments. The four lowest-SSIM test pairs (indices 221, 65 and 42 in Style 2 and 517 in Style 1; SSIM 0.17–0.21) all have real sketches with dense individual strokes (hair strands, shading on the cheeks and neck), whereas the generated sketch is a soft blur that keeps

Missing figure:
task4/style_comparison.png

Fig. 18. Same photographs under the three style conditions. Rows: photograph, real sketch, Style 1, Style 2, Style 3.

Missing figure:
task4/examples_1.png

Missing figure:
task4/examples_2.png

Fig. 19. Task 4 test examples (12 pairs): photograph, real sketch and generated sketch.

TABLE XIV
EFFECT OF THE RECONSTRUCTION WEIGHT ON THE TEST SET (ONE RUN EACH).

Metric	$\lambda_{L1} = 200$ (selected)	$\lambda_{L1} = 50$
L1	0.1093	0.1082
PSNR (dB)	15.27	15.37
SSIM	0.486	0.484
Style 2 PSNR (dB)	12.38	12.37
Style-match rate	81.1%	81.1%

Missing figure:
task4/failure_cases.png

Fig. 20. Task 4 failure cases: the lowest-SSIM test pairs.

the dark hair mass, eyes and mouth but no strokes; in the Style 1 case the output is very faint, almost a blank face.

Only the generator is exported to ONNX and used by the application.

VIII. ONNX EXPORT AND VERIFICATION

All seven inference models are exported to ONNX (opset with dynamic batch size) and compared with their PyTorch originals on identical inputs, for batches and for a single image (Table XV). The largest absolute difference is 1.3×10^{-6} against tolerances of 10^{-3} for images and 10^{-4} for classifier probabilities; the exported classifier agrees with PyTorch on every arg-max, and the exported soft-MoE weights sum to one. Latency is measured on CPU through ONNX Runtime.

IX. APPLICATION DESIGN AND DEPLOYMENT

A. Design in Google Stitch

The interface was first designed in Google Stitch as four mobile screens, one per workspace, with a shared bottom navigation (Fig. 21). Checking the first design against the real models showed several mismatches: expert names that did not exist (“Super-Resolution”, “Deblur Transformer”), controls without a model behind them (stroke-intensity sliders, a temperature slider, a vector export), invented metrics and hardware readouts, and blank panels. The design was revised accordingly (Fig. 22): the routing screens show the real four branches (clean/identity, salt-and-pepper, blur, occlusion), the soft-MoE screen shows the temperature as a fixed value, the sketch screen offers exactly Style 1, 2 and 3, and uploads are limited to PNG, JPEG or WEBP up to 10MB. A few

TABLE XV
ONNX MODELS: SIZE, MAXIMUM ABSOLUTE DIFFERENCE TO PYTORCH, AND CPU LATENCY PER IMAGE.

Model	Size (MB)	Max. abs. diff.	ms/image
Task 1 universal	10.2	1.1×10^{-6}	18.0
Task 2 classifier	1.18	2.4×10^{-9}	4.0
Task 2 salt specialist	10.07	6.9×10^{-7}	19.7
Task 2 blur specialist	10.07	5.4×10^{-7}	19.8
Task 2 occlusion specialist	10.07	8.3×10^{-7}	28.8
Task 3 soft MoE (full pipeline)	31.41	7.2×10^{-7}	74.8
Task 4 generator	66.22	1.3×10^{-6}	30.6

values in the revised mock-ups are illustrative (for example the severity parameters and latencies); the built application shows the values returned by the backend.

B. Architecture

The frontend (React 18, Vite and Tailwind CSS) is built in a Node container and served by nginx, which also proxies /api/ to a FastAPI backend (Python 3.12) that runs the seven ONNX models with ONNX Runtime on CPU. The backend validates every upload (type, a 10 MB size limit and decodability; errors return 400, 413, 415, 422 or 503 with a readable message), resizes images to 128×128, applies the requested corruption with the same functions and fixed severities as the evaluation (seeded and reproducible), runs inference and returns the input, restored image, optional clean original, the corruption settings, PSNR when a clean reference exists, routing probabilities or weights, and timings. The endpoints are GET /api/health, GET /api/samples, POST /api/universal, POST /api/hard (predicted or oracle routing), POST /api/soft and POST /api/sketch. Thirteen automated backend tests pass, including a check that the seeded corruption settings match the specification table. Model files are not stored in Git; a script downloads them from a shared folder.

C. Workspaces and deployment

The four workspaces (Fig. 23) accept an uploaded image, a bundled sample or (for sketches) a webcam capture. In *Universal Restoration* the user uploads an already corrupted image or applies a corruption and severity to a clean sample, and sees the input, the restored output, the settings and the inference time. *Hard-Routed Restoration* shows the four classifier probabilities, the predicted corruption,

Missing figure: stitch_v1/universal.jpeg
Missing figure: stitch_v1/hard_routed.jpeg
Missing figure: stitch_v1/soft_moe.jpeg
Missing figure: stitch_v1/face_to_sketch.jpeg

Fig. 21. Original Stitch design (first version): Universal, Hard-Routed, Soft-MoE and Face-to-Sketch screens.

Missing figure: stitch_v2/universal.jpeg
Missing figure: stitch_v2/hard_routed.jpeg
Missing figure: stitch_v2/soft_moe.jpeg
Missing figure: stitch_v2/face_to_sketch.jpeg

Fig. 22. Revised Stitch design after aligning labels and controls with the real models.

the selected expert (or “identity bypass”), an oracle/predicted switch and the timing split. *Soft MoE* shows the four routing weights, marks the dominant expert and lists the contributing branches, with the temperature displayed as fixed. *Face-to-Sketch* shows the photograph and sketch side by side for Style 1, 2 or 3 and offers a PNG download; every restoration result can also be downloaded. The whole system starts with one command, `docker compose up --build`, after `python scripts/download_models.py`, and is opened at `http://localhost:8080`; a container health check on the backend delays the frontend until the models are loaded.

D. Experiment tracking and the Optuna record

Training and tuning runs were logged to Weights & Biases (<https://forge.coreweave.com/wandb/i230788-fast-nuces/genai-a1-restoration>): hyperparameters, per-epoch losses and metrics (including per-condition validation PSNR and SSIM, classifier per-class F1, routing weights and the separate GAN losses), checkpoints as artifacts and sample grids on fixed validation images. Table XVI lists every Optuna study with its true completed and pruned counts; the Optuna databases and trial tables are in the repository (`configs/`). The four controlled ablation runs of Task 1 were run without W&B logging, so their results are reported from the training output only.

X. LIMITATIONS

- **Capacity versus fidelity.** A genuine compression bottleneck cannot reproduce fine detail: even clean images are restored at only about 22 dB by the universal model, so restoration can be worse than the input for mild corruption (blur) and for clean images. The identity bypass (Task 2) and the identity branch (Task 3) are the remedy, and even the soft MoE stays below the input for the mildest blur.

TABLE XVI
ALL OPTUNA STUDIES (TPE SAMPLER, MEDIAN PRUNER, SEED 42).

Study	Trials	Compl.	Pruned	Best obj.
Task 1, v1 (flat vector)	20	8	12	0.4105
Task 1, v2 (spatial latent)	20	8	12	0.5442
Task 1, v3 (+ skip option)	24	9	15	0.6028
Classifier (macro-F1)	30	6	24	0.9986
Specialists, v1	13	4	8 (1 failed)	0.3896
Specialists, v2	8	4	4	0.5366
Specialists, v3	12	6	6	0.5999
Soft MoE	10	10	0	0.8114
Task 4 cGAN (val. SSIM)	12	4	8	0.5006

- **Single seeds and short searches.** Every number comes from one run with seed 42. Differences of a few thousands (for example skip versus no-skip, 0.639 versus 0.646, or $\lambda_{L1} = 50$ versus 200) are within plausible seed noise. The Optuna studies are small (8–30 trials, short trials, a median pruner), several optima lay on the edge of the searched range, and the Task 3 search was nearly flat.
- **Proxy search and missing controls.** The specialist architecture was found with one autoencoder trained on all three corruptions, not per corruption, and there is no no-skip control at convergence for the specialists.
- **Test-set use.** Earlier model versions (v1, v2) were also evaluated on the test set before v3 was designed; all selection used validation data only.
- **Metrics.** PSNR is capped at 50 dB, which inflates the overall averages whenever clean inputs are left unchanged; per-condition rows are the fair comparison.
- **Face-to-sketch quality.** The sketches are blurry, with weak adversarial signal, memorisation of 899 training pairs and loosely aligned targets; the official test split is imbalanced (619/381/46), so Style 3 is barely evaluated.

Missing figure: app/universal.png
Missing figure: app/hard_routed.png
Missing figure: app/soft_moe.png
Missing figure: app/face_to_sketch.png

Fig. 23. Screenshots of the running application: Universal, Hard-Routed, Soft-MoE and Face-to-Sketch workspaces.

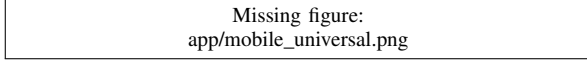


Fig. 24. Responsive (mobile-width) layout of the Universal workspace.

- **Application.** Inference runs on CPU; the soft MoE takes about 75 ms per image in the benchmark (single images in the container can be slower on a first request). The webcam requires a secure context (localhost or HTTPS), and the model files must be downloaded separately from the repository.

XI. CONCLUSION

We compared three restoration designs and a style-conditioned cGAN under identical data splits and metrics, and delivered them as one containerised application. A real compression bottleneck limits a single universal autoencoder to SSIM 0.735; routing each image to a specialist raised this to 0.804 with a 99.8%-accurate classifier, and a jointly fine-tuned soft mixture-of-experts reached 26.91 dB and 0.828, with the gains concentrated at low corruption severity where the gate learned to mix in the identity branch. All experts remained active and specific, and the gate traded some classification accuracy (88.4%) for better restoration. The design evolved through controlled experiments (vector versus spatial bottleneck, narrow skip versus no skip, reconstruction weight), several of which overturned the initial expectation: a skip connection helped in short runs but not at convergence, and lowering the L1 weight did not sharpen the generated sketches. The face-to-sketch generator responds clearly to the style condition but produces blurry sketches (test SSIM 0.486), the weakest result. All models match their PyTorch versions after ONNX export (maximum difference 1.3×10^{-6}) and run behind a FastAPI/React application started with Docker Compose. Future work includes a perceptual or feature-matching loss and data augmentation for the cGAN, per-severity experts, and multi-seed evaluation of the architectural comparisons.

APPENDIX A

AI-ASSISTED DEVELOPMENT STATEMENT

Tools used. Anthropic’s Claude was used for project planning, drafting data-pipeline, training, Optuna, evaluation, ONNX-export and backend code, analysing result tables, and drafting this report’s LaTeX. Google Stitch was used for the interface design. An AI coding agent (Google Antigravity)

generated the React/Tailwind frontend from a written specification that listed the exact backend contract and design screens. Weights & Biases, Optuna, PyTorch and ONNX Runtime were used as libraries.

How outputs were tested and corrected.

- Every training script was first run as a short smoke test; checks printed class counts per balanced batch, verified that the validation manifest matches the stored split, and confirmed determinism of manifest-based corruptions.
- Diagnostics found real problems that the generated code had not anticipated: an FS2K style/folder mismatch (the folder name is not the style label), an empty Drive data folder (data and manifests were regenerated and re-verified), an overwritten checkpoint (restored from a backup), a shell syntax error and a working-directory error in notebook cells, and a .gitignore rule that silently excluded a results folder.
- The AI’s first recommendation from a 15-epoch ablation (use a narrow skip connection) was tested again at 100 epochs, and the control reversed it; the no-skip model was adopted.
- An AI-suggested hypothesis (that a large L_1 weight blurs the sketches) was tested with a controlled retraining and rejected.
- ONNX models were compared with PyTorch outputs numerically (Table XV); 13 backend tests were run; the generated frontend was built with `npm ci` and `npm run build`, exercised in the browser for every workspace, and its screenshots were checked.
- All tables in this report were generated from the result files in the repository (`reports/`) and failure-case descriptions were checked against the saved figures.

Reused components: PyTorch, torchvision datasets (Oxford-IIIT Pet download), the `pytorch-msssim` SSIM implementation, Optuna, Weights & Biases, ONNX Runtime, FastAPI, React, Vite and Tailwind CSS, with the cited architectures (U-Net, PatchGAN, FiLM) re-implemented. I take responsibility for the correctness of the submission and can explain and modify any component.

REFERENCES

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proc. ICML*, 2008.

- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [3] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in *Proc. CVPR*, 2016.
- [4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015.
- [5] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Trans. Comput. Imaging*, vol. 3, no. 1, pp. 47–57, 2017.
- [6] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [7] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Comput.*, vol. 3, no. 1, pp. 79–87, 1991.
- [8] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
- [9] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, 2022.
- [10] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. NeurIPS*, 2014.
- [11] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv:1411.1784, 2014.
- [12] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. CVPR*, 2017.
- [13] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI*, 2018.
- [14] D.-P. Fan *et al.*, "Facial-sketch synthesis: A new challenge," *Mach. Intell. Res.*, vol. 19, pp. 257–287, 2022.
- [15] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *Proc. CVPR*, 2012.
- [16] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015.
- [17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [18] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019.
- [19] L. Biewald, "Experiment tracking with Weights and Biases," 2020. [Online]. Available: <https://www.wandb.com/>
- [20] ONNX Runtime developers, "ONNX Runtime," 2021. [Online]. Available: <https://onnxruntime.ai>